

claude-windows / 6a82308f-69d6-43a8-9ea4-d8eaf497253d

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\subagents\workflows\wf_736be611-fb5\agent-a63714010d6068e8e.jsonl`
- SHA-256: `8001d27d00f603b112f0dd78dc5610edc05432f44b02c9274e1d9dddb9237d7f`
- Source modified: `2026-06-16T15:16:26+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_736be611-fb5`
- session_id: `6a82308f-69d6-43a8-9ea4-d8eaf497253d`
```

Transcript

1. user (2026-06-16T15:14:30.300Z)

Adversarially VERIFY this finding from the real-footage rally-detection validation. Try hard to REFUTE or show it is overstated, using ONLY the data (you MAY Read output/local_vs_gemini_6.json). Default to holds=false if the numbers do not clearly support it.

CLAIM (trajectory-signal): The same windowing behaves differently across clips because the active-frame rule keys on 'shuttle moving' not 'rally', and clips differ in how often the shuttle goes invisible/slow between rallies ? that invisibility/slowdown is the ONLY thing that ends or splits a window.

EVIDENCE CITED: Code: a window's end = group[-1][0]/fps, the last in-flight frame (tracknet_runner.py:305); prev resets to None on any invisible frame (line 285), and active requires velocity>velocity_thresh=0.02 (line 292). W1's cap splits ONLY at the largest internal inactivity gap >= min_split_gap and leaves a gap-less group intact (_split_overlong_groups, lines 358-364). So a clip with frequent shuttle-invisible/dead frames between points gets natural cut points (mahadevpura-2 W1 mean 13.8s, count_ratio 1.03, near-perfect 40 vs 39); a clip with continuous low-level shuttle visibility/motion has no gap to cut on and stays one blob (mahadevpura-1 W1 mean 86.9s, n=2 vs Gemini 9, mIoU 0.0).

REAL-FOOTAGE VALIDATION DATA (6 Kushagra/Yash June-12 clips, 1080p proxies, fully-LOCAL no-AI WASB

windowing vs Gemini. Gemini = strong REFERENCE, NOT ground truth ? these 6 have no golden labels EXCEPT mahadevpura-2 which the owner hand-labeled, see GROUND-TRUTH below).

Rally COUNT (Gemini | local baseline | local W1 cap=12 | local W1+W2 mc=1):

```
GX010128: 39 | 15 | 78 | 75
GX010129: 27 | 2 | 50 | 47
GX020125: 11 | 6 | 18 | 15
mahadevpura-0: 40 | 19 | 62 | 43
mahadevpura-1: 9 | 1 | 2 | 2
mahadevpura-2: 39 | 5 | 40 | 40
```

```
AGGREGATE: Gemini 165 | baseline 48 (mean window 62.9s, 27 merge-groups, mIoU 0.23) |
           W1 250 (mean 10.8s, 10 merge-groups, mIoU 0.39) |
           W1+W2 222 (mean 11.5s, 10 merge-groups, mIoU 0.39, 6 gemini-only "misses")
```

Per-video mean window (W1): GX010128 7.8s, GX010129 9.3s, GX020125 14.1s, mahadevpura-0 10.4s, mahadevpura-1 86.9s (the continuously-active blob W1 cannot cap ? no inactivity gap), mahadevpura-2 13.8s. Gemini means ~5.7-7.2s.

GROUND-TRUTH (mahadevpura-2, 40 owner-labeled rallies, mean 5.7s, verified frame-aligned):

```
F1@0.5 / F1@0.25 / mean-IoU: Gemini 0.557/0.835/0.55 ; local W1+W2 0.150/0.500/0.31 ;
local W1+W2+hard-cap 0.296/0.722/0.50 (best local). baseline 0.000/0.000/0.03.
BOUNDARY ERROR (matched rallies, seconds): Gemini |?start| med 1.46 |?end| med 1.19 ;
local-hardcap |?start| med 1.74 (~= Gemini!) |?end| med 3.12 (>> Gemini).
=> local's rally-START detection is ~at par with Gemini; the gap is the rally-END.
```

IoU<->time: for a 5.7s rally, tIoU 0.5 tolerates only ~+-1.9s shift (the ~-1-1.5s service window

nearly exhausts it -> tIoU 0.5 over-penalizes; owner wants a +-2s tolerance metric).

PRIOR GOLDEN DELTAS (n=6, then n=7 incl. mahadevpura-2): baseline F1 0.300 -> W1 0.439 (+0.139, 6/0)

-> W1+W2 0.457 (+0.019, 6/0); hard-cap golden-neutral on n=6 (+0.006 n.s.) but became BEST on n=7

(continuously-active clips). All flags default-OFF, promoted only on measured LOVO lift + sign-test.

WINDOWING KNOBS: a frame is "active" iff shuttle visible AND per-second-normalized velocity > velocity_thresh(0.02); active frames within merge_gap(2.0s) coalesce; min_window(2.0s). W1 = max_window_duration cap(12s) split at the largest internal inactivity gap. W1.5 hard-cap = midpoint-chop when no gap. W2 = net-crossings gate (drop windows with <1 net crossing).

2. assistant (2026-06-16T15:14:34.184Z)

I'll verify this claim by examining the actual code and data. Let me start by reading the relevant source file and the validation data.

3. user (2026-06-16T15:14:35.419Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\w2-safeguard.

4. user (2026-06-16T15:14:35.505Z)

backend\pipeline\detectors\tracknet_runner.py

5. assistant (2026-06-16T15:14:38.029Z)

The data file isn't at the expected path. Let me find it.

6. user (2026-06-16T15:14:38.603Z)

No files found

7. user (2026-06-16T15:14:39.573Z)

```
270         A frame is "active" when the shuttle is visible AND its inter-frame
271         displacement (normalised by frame width, per second) exceeds
272         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
273         ``merge_gap`` seconds of each other are grouped into one window; short
274         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
275         shape feeding the AI-handoff phase is identical.
276
277         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
278         the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
279         and a single window swallows several rallies + the dead time between them. When
set, any
280         window longer than this is recursively SPLIT at its largest internal inactivity
gap (the
281         longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
282         that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
283         ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
284         drop, and it is config-gated default-OFF until measured on the golden set.
285         """
286         if fps <= 0:
```

```

287         fps = 30.0
288     if frame_width <= 0:
289         frame_width = 1280.0
290
291     # Compute per-frame normalised velocity from the last *visible* point.
292     active = [] # (frame_idx, velocity)
293     prev: Optional[TrajectoryPoint] = None
294     for p in points:
295         if not p.visible:
296             prev = None # break the trajectory; absent shuttle = not in flight
297             continue
298         if prev is not None:
299             dframes = p.frame - prev.frame
300             if dframes > 0:
301                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
302                 velocity = (dist / frame_width) * (fps / dframes)
303                 if velocity > velocity_thresh:
304                     active.append((p.frame, velocity))
305             prev = p
306
307     if not active:
308         return []
309
310     max_gap_frames = int(merge_gap * fps)
311
312     def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
313         vels = [v for _, v in group]
314         return {
315             "start": group[0][0] / fps + chunk_start,
316             "end": group[-1][0] / fps + chunk_start,
317             "active_frame_count": len(group),
318             "peak_velocity": max(vels),
319             "mean_velocity": sum(vels) / len(vels),
320             "track_id_count": 1, # single shuttle; kept for window-shape parity
321             "chunk_index": chunk_index,
322         }
323
324     groups: List[List[Tuple[int, float]]] = []
325     group = [active[0]]
326     for af in active[1:]:
327         if af[0] - group[-1][0] <= max_gap_frames:
328             group.append(af)
329         else:
330             groups.append(group)
331             group = [af]
332     groups.append(group)
333
334     if max_window_duration > 0:
335         groups = TrackNetRunner._split_overlong_groups(
336             groups, int(max_window_duration * fps), int(min_split_gap * fps))
337
338     windows = [_finalize(g) for g in groups]
339     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
340
341     @staticmethod
342     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
343     int,
344                                min_split_frames: int) -> List[List[Tuple[int, float]]]:
345         """Recursively split any active-frame group longer than ``max_win_frames`` at
346         its largest
347         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
348         likely rally
349         boundary. Splitting only subdivides existing groups (never merges/extends), so
350         it cannot
351         invent or drop coverage; a group with no gap ? the floor is left intact (a

```

```

genuinely long
348         rally). Iterative worklist to bound stack depth."""
349         if max_win_frames <= 0:
350             return groups
351         out: List[List[Tuple[int, float]]] = []
352         work = list(groups)
353         while work:
354             g = work.pop()
355             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
356                 out.append(g)
357                 continue
358             best_i, best_gap = -1, -1
359             for i in range(1, len(g)):
360                 gap = g[i][0] - g[i - 1][0]
361                 if gap > best_gap:
362                     best_gap, best_i = gap, i
363             if best_i < 0 or best_gap < min_split_frames:
364                 out.append(g) # too long but no real dead-time gap ? keep as one rally
365                 continue
366             work.append(g[:best_i])
367             work.append(g[best_i:])
368         out.sort(key=lambda grp: grp[0][0])
369         return out
370

```

8. assistant (2026-06-16T15:14:42.829Z)

The data file `output/local\_vs\_gemini\_6.json` does not exist in this worktree. Let me search more broadly and confirm the code lines cited.

9. user (2026-06-16T15:14:43.374Z)

eval_baselines\gemini_wrapper_2026-06-10.json

10. user (2026-06-16T15:14:43.391Z)

```

output\wasb\GX010128__db84f3e65318_wasb_wasbcache\det_raw.jsonl
output\wasb\GX010128__db84f3e65318_wasb_wasbcache\manifest.json
output\wasb\GX010128__db84f3e65318_wasb.csv
output\val_reels\GX010128 - Highlights.mp4
output\highlights_work\GX010128 - run.log
output\val_reels\GX010128 - run.log
output\wasb\GX010129__99a3e73afde3_wasb_wasbcache\det_raw.jsonl
output\wasb\GX010129__99a3e73afde3_wasb_wasbcache\manifest.json
output\wasb\GX010129__99a3e73afde3_wasb.csv
output\val_reels\GX010129 - Highlights.mp4
output\highlights_work\GX010129 - run.log
output\val_reels\GX010129 - run.log
output\wasb\mahadevpura-0__6460c2fc7647_wasb_wasbcache\det_raw.jsonl
output\wasb\mahadevpura-0__6460c2fc7647_wasb_wasbcache\manifest.json
output\wasb\mahadevpura-0__6460c2fc7647_wasb.csv
output\val.db
output\val_reels\mahadevpura-0 - Highlights.mp4
output\highlights_work\mahadevpura-0 - run.log
output\highlights_work\batch_run.log
output\val_reels\mahadevpura-0 - run.log
output\val_reels\batch_run.log
output\val_extract.log

```

11. user (2026-06-16T15:14:46.285Z)

```

C:/Users/avidu/Projects/badminton-highlight-indexer/output/local_vs_gemini_6.json
C:/Users/avidu/Projects/badminton-highlight-indexer/output/local_vs_gemini_6.md
C:/Users/avidu/Projects/badminton-highlight-indexer/output/local_vs_gemini_report.md

```

C:/Users/avidu/Projects/badminton-highlight-indexer/scratch/compare_local_vs_gemini.py

---worktree---

total 292

```
drwxr-xr-x 1 avidu 197609      0 Jun 16 20:34 .
drwxr-xr-x 1 avidu 197609      0 Jun 16 11:24 ..
drwxr-xr-x 1 avidu 197609      0 Jun 16 11:14 chunks
drwxr-xr-x 1 avidu 197609      0 Jun 16 20:38 highlights_work
-rw-r--r-- 1 avidu 197609 212992 Jun 16 20:34 val.db
-rw-r--r-- 1 avidu 197609  61195 Jun 16 20:38 val_extract.log
drwxr-xr-x 1 avidu 197609      0 Jun 16 20:38 val_reels
drwxr-xr-x 1 avidu 197609      0 Jun 16 20:34 wasb
```

---main output---

total 5048098

```
drwxr-xr-x 1 avidu 197609      0 Jun 16 20:39 .
drwxr-xr-x 1 avidu 197609      0 Jun 16 09:42 ..
-rw-r--r-- 1 avidu 197609 1207870728 Jun 13 15:03 GX010125_highlights.mp4
-rw-r--r-- 1 avidu 197609 2083468872 Jun 13 20:19 GX010125_highlights_min5.mp4
drwxr-xr-x 1 avidu 197609      0 Jun 14 10:48 GX010125_run
-rw-r--r-- 1 avidu 197609 1496443316 Jun 13 22:30 KushagraAviHighlight-Trailor.mp4
-rw-r--r-- 1 avidu 197609 125200877 Jun 10 16:31 MahadevpuraSingles_highlights.mp4
-rw-r--r-- 1 avidu 197609    2253 Jun 14 16:07 ab_experiment_n6.json
-rw-r--r-- 1 avidu 197609    9016 Jun  8 18:24 adarsh_avi_full_vs_human.csv
-rw-r--r-- 1 avidu 197609   131699 Jun 14 11:54 adarsh_avi_singles_golden_features.json
-rw-r--r-- 1 avidu 197609     561 Jun  9 18:14 adarsh_avi_vs_human.csv
-rw-r--r-- 1 avidu 197609    1721 Jun  6 21:23 adarsh_baseline_gated.csv
-rw-r--r-- 1 avidu 197609    2023 Jun  6 21:15 adarsh_baseline_segments.csv
-rw-r--r-- 1 avidu 197609    1628 Jun  6 22:40 adarsh_gemini_fullvideo.csv
-rw-r--r-- 1 avidu 197609    1670 Jun  6 22:21 adarsh_gemini_refined.csv
-rw-r--r-- 1 avidu 197609     269 Jun  6 22:03 adarsh_gemini_smoke.csv
-rw-r--r-- 1 avidu 197609    1327 Jun  6 21:29 adarsh_min5s_segments.csv
-rw-r--r-- 1 avidu 197609    2895 Jun  6 20:54 adarsh_tuned_segments.csv
-rw-r--r-- 1 avidu 197609   36864 May 19 23:07 badminton_indexer.db
drwxr-xr-x 1 avidu 197609      0 Jun 16 01:38 chunks
-rw-r--r-- 1 avidu 197609   3979353 Jun 11 20:43 e2e_reel.mp4
-rw-r--r-- 1 avidu 197609    2481 Jun 11 20:43 e2e_server_log.txt
-rw-r--r-- 1 avidu 197609    67688 Jun 14 13:35 fusion_golden.log
drwxr-xr-x 1 avidu 197609      0 Jun 14 16:06 gateA_served_verify
-rw-r--r-- 1 avidu 197609    62555 Jun 14 11:54 gbaaddy_golden_features.json
-rw-r--r-- 1 avidu 197609    5645 Jun  8 21:56 gbaaddy_vs_human.csv
-rw-r--r-- 1 avidu 197609    6756 Jun 13 20:19 gx010125_min5_stitch.log
-rw-r--r-- 1 avidu 197609   108744 Jun 13 15:03 gx010125_run.log
-rw-r--r-- 1 avidu 197609    35165 Jun 12 22:29 gx010125_transcode.log
drwxr-xr-x 1 avidu 197609      0 Jun 15 23:42 highlights_work
-rw-r--r-- 1 avidu 197609    2135 Jun 16 19:45 human_lovo_manifest.json
-rw-r--r-- 1 avidu 197609    1766 Jun 10 12:56 human_lovo_manifest.json.bak
-rw-r--r-- 1 avidu 197609 160570312 Jun 10 16:46 kushagra_1280.mp4
-rw-r--r-- 1 avidu 197609    2659 Jun 10 17:01 kushagra_1280.report.json
-rw-r--r-- 1 avidu 197609    1166 Jun 10 17:01 kushagra_1280.report.md
-rw-r--r-- 1 avidu 197609     480 Jun 10 17:01 kushagra_1280.summary.md
drwxr-xr-x 1 avidu 197609      0 Jun 16 01:40 kushagra_highlights_work
-rw-r--r-- 1 avidu 197609   16195 Jun 14 11:54 kushagra_singles_golden_features.json
-rw-r--r-- 1 avidu 197609    3200 Jun  8 18:42 kushagra_vs_human.csv
-rw-r--r-- 1 avidu 197609   64537 Jun 14 11:54 largetest_doubles_golden_features.json
-rw-r--r-- 1 avidu 197609    6120 Jun  8 21:03 largetestvideo_vs_human.csv
-rw-r--r-- 1 avidu 197609     370 Jun  6 22:40 local_calib_manifest.json
-rw-r--r-- 1 avidu 197609   86016 Jun 16 02:27 local_compare.db
-rw-r--r-- 1 avidu 197609     662 Jun  6 22:51 local_rally_model.json
-rw-r--r-- 1 avidu 197609    9205 Jun 16 20:39 local_vs_gemini_6.json
-rw-r--r-- 1 avidu 197609    1873 Jun 16 20:39 local_vs_gemini_6.md
-rw-r--r-- 1 avidu 197609    5719 Jun 16 01:57 local_vs_gemini_report.md
-rw-r--r-- 1 avidu 197609    1719 Jun 16 19:30 mahadevpura_2_rallies.csv
-rw-r--r-- 1 avidu 197609    3241 Jun 16 19:45 mahadevpura_2_golden_features.json
-rw-r--r-- 1 avidu 197609    1451 Jun 10 18:07 mahadevpura_gemini_refined.csv
-rw-r--r-- 1 avidu 197609   43410 Jun 14 11:54 mahadevpura_singles_golden_features.json
-rw-r--r-- 1 avidu 197609   90194321 Jun 11 11:21 mahadevpura_smoke_180s.mp4
```

```

-rw-r--r-- 1 avidu 197609      3214 Jun 11 18:53 mahadevpura_smoke_180s.report.json
-rw-r--r-- 1 avidu 197609      1565 Jun 11 18:53 mahadevpura_smoke_180s.report.md
-rw-r--r-- 1 avidu 197609        486 Jun 11 18:53 mahadevpura_smoke_180s.summary.md
-rw-r--r-- 1 avidu 197609      3438 Jun  9 10:40 mahadevpura_vs_human.csv
-rw-r--r-- 1 avidu 197609       1934 May 26 04:04 phase3_probe.json
-rw-r--r-- 1 avidu 197609     114688 May 26 03:53 phase3_yolo.db
-rw-r--r-- 1 avidu 197609     141594 Jun 10 16:21 ramp_probe.mp4
-rw-r--r-- 1 avidu 197609      47104 Jun 11 18:53 smoke4_log.txt
-rw-r--r-- 1 avidu 197609     245760 Jun 16 01:38 sports_indexer.db
-rw-r--r-- 1 avidu 197609      24576 May 20 12:04 test_indexer.db
-rw-r--r-- 1 avidu 197609        350 Jun  6 22:30 testlarge_gemini_fullvideo.csv
-rw-r--r-- 1 avidu 197609        270 Jun  6 22:24 testlarge_gemini_rallies.csv
-rw-r--r-- 1 avidu 197609     10505 Jun 14 11:54 testlarge_short_golden_features.json
-rw-r--r-- 1 avidu 197609       1171 Jun  8 21:56 testlarge_vs_human.csv
-rw-r--r-- 1 avidu 197609     22395 Jun 13 22:28 trim70.log
drwxr-xr-x 1 avidu 197609         0 Jun 11 20:43 uploads
drwxr-xr-x 1 avidu 197609         0 Jun 15 21:47 wasb
drwxr-xr-x 1 avidu 197609         0 Jun 16 00:04 wasb_sxs
-rw-r--r-- 1 avidu 197609      1526 Jun  6 20:03 wasb_tuned_segments.csv
-rw-r--r-- 1 avidu 197609      1185 Jun  6 20:03 wasb_tuned_segments_baseline.csv
-rw-r--r-- 1 avidu 197609      2997 Jun  6 20:03 wasb_tuned_segments_vs_golden.csv

```

12. assistant (2026-06-16T15:14:49.436Z)

The data file is in the main repo at `C:/Users/avidu/Projects/badminton-highlight-indexer/output/local\_vs\_gemini\_6.json`. Let me read it.

13. user (2026-06-16T15:14:49.789Z)

```

1      [
2      {
3          "name": "GX010128",
4          "vid": "db84f3e65318",
5          "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\w2-safeguard\\output\\wasb\\GX010128__db84f3e65318_wasb.csv",
6          "n_gem": 39,
7          "play_gem": 228.0,
8          "mean_gem": 5.846153846153846,
9          "variants": {
10             "baseline": {
11                 "n": 15,
12                 "play": 711.4614614614616,
13                 "mean_dur": 47.430764097430774,
14                 "max_dur": 133.2499165832499,
15                 "count_ratio": 0.38461538461538464,
16                 "matched": 4,
17                 "local_only": 1,
18                 "gemini_only": 0,
19                 "splits": 0,
20                 "merges": 10,
21                 "mean_iou": 0.3564105364413525,
22                 "gem_rallies_in_merges": 35
23             },
24             "w1": {
25                 "n": 78,
26                 "play": 605.1051051051056,
27                 "mean_dur": 7.757757757757765,
28                 "max_dur": 16.616616616616625,
29                 "count_ratio": 2.0,
30                 "matched": 35,
31                 "local_only": 34,
32                 "gemini_only": 0,
33                 "splits": 4,
34                 "merges": 0,
35                 "mean_iou": 0.5294639963484731,

```

```

36         "gem_rallies_in_merges": 0
37     },
38     "w1+w2": {
39         "n": 75,
40         "play": 594.494494494495,
41         "mean_dur": 7.926593259926601,
42         "max_dur": 16.616616616616625,
43         "count_ratio": 1.9230769230769231,
44         "matched": 35,
45         "local_only": 31,
46         "gemini_only": 0,
47         "splits": 4,
48         "merges": 0,
49         "mean_iou": 0.5294639963484731,
50         "gem_rallies_in_merges": 0
51     }
52 },
53     "status": "ok"
54 },
55 {
56     "name": "GX010129",
57     "vid": "99a3e73afde3",
58     "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\w2-safeguard\\output\\wasb\\GX010129__99a3e73afde3_wasb.csv",
59     "n_gem": 27,
60     "play_gem": 195.6,
61     "mean_gem": 7.2444444444444445,
62     "variants": {
63         "baseline": {
64             "n": 2,
65             "play": 534.4177510844178,
66             "mean_dur": 267.2088755422089,
67             "max_dur": 522.6893560226894,
68             "count_ratio": 0.07407407407407407,
69             "matched": 0,
70             "local_only": 1,
71             "gemini_only": 0,
72             "splits": 0,
73             "merges": 1,
74             "mean_iou": 0.0,
75             "gem_rallies_in_merges": 27
76         },
77         "w1": {
78             "n": 50,
79             "play": 463.7137137137137,
80             "mean_dur": 9.274274274274273,
81             "max_dur": 27.84451117784451,
82             "count_ratio": 1.8518518518518519,
83             "matched": 23,
84             "local_only": 18,
85             "gemini_only": 0,
86             "splits": 4,
87             "merges": 0,
88             "mean_iou": 0.4819655005521238,
89             "gem_rallies_in_merges": 0
90         },
91         "w1+w2": {
92             "n": 47,
93             "play": 447.0303636970303,
94             "mean_dur": 9.511284333979368,
95             "max_dur": 27.84451117784451,
96             "count_ratio": 1.7407407407407407,
97             "matched": 22,
98             "local_only": 16,
99             "gemini_only": 1,

```

```

100         "splits": 4,
101         "merges": 0,
102         "mean_iou": 0.48514996335815863,
103         "gem_rallies_in_merges": 0
104     }
105 },
106     "status": "ok"
107 },
108 {
109     "name": "GX020125",
110     "vid": "407fa7e7e9f1",
111     "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\local-noai\\output\\wasb\\GX020125__407fa7e7e9f1_wasb.csv",
112     "n_gem": 11,
113     "play_gem": 71.0,
114     "mean_gem": 6.454545454545454,
115     "variants": {
116         "baseline": {
117             "n": 6,
118             "play": 280.6806806806807,
119             "mean_dur": 46.78011344678012,
120             "max_dur": 82.66599933266599,
121             "count_ratio": 0.5454545454545454,
122             "matched": 1,
123             "local_only": 2,
124             "gemini_only": 0,
125             "splits": 0,
126             "merges": 3,
127             "mean_iou": 0.40270862800565765,
128             "gem_rallies_in_merges": 10
129         },
130         "w1": {
131             "n": 18,
132             "play": 254.03737070403744,
133             "mean_dur": 14.113187261335414,
134             "max_dur": 40.05672339005673,
135             "count_ratio": 1.6363636363636365,
136             "matched": 9,
137             "local_only": 8,
138             "gemini_only": 0,
139             "splits": 0,
140             "merges": 1,
141             "mean_iou": 0.4329964346464835,
142             "gem_rallies_in_merges": 2
143         },
144         "w1+w2": {
145             "n": 15,
146             "play": 246.5131798465132,
147             "mean_dur": 16.434211989767547,
148             "max_dur": 40.05672339005673,
149             "count_ratio": 1.3636363636363635,
150             "matched": 9,
151             "local_only": 5,
152             "gemini_only": 0,
153             "splits": 0,
154             "merges": 1,
155             "mean_iou": 0.4329964346464835,
156             "gem_rallies_in_merges": 2
157         }
158     },
159     "status": "ok"
160 },
161 {
162     "name": "mahadevpura-0",
163     "vid": "6460c2fc7647",

```



```

164         "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\w2-safeguard\\output\\wasb\\mahadevpura-0__6460c2fc7647_wasb.csv",
165         "n_gem": 40,
166         "play_gem": 262.0,
167         "mean_gem": 6.55,
168         "variants": {
169             "baseline": {
170                 "n": 19,
171                 "play": 711.5615615615616,
172                 "mean_dur": 37.45060850324008,
173                 "max_dur": 134.13413413413414,
174                 "count_ratio": 0.475,
175                 "matched": 6,
176                 "local_only": 4,
177                 "gemini_only": 0,
178                 "splits": 0,
179                 "merges": 9,
180                 "mean_iou": 0.3446682872977475,
181                 "gem_rallies_in_merges": 34
182             },
183             "w1": {
184                 "n": 62,
185                 "play": 646.1127794461132,
186                 "mean_dur": 10.421173862034085,
187                 "max_dur": 47.66433099766434,
188                 "count_ratio": 1.55,
189                 "matched": 34,
190                 "local_only": 22,
191                 "gemini_only": 0,
192                 "splits": 2,
193                 "merges": 2,
194                 "mean_iou": 0.4928109196809172,
195                 "gem_rallies_in_merges": 4
196             },
197             "w1+w2": {
198                 "n": 43,
199                 "play": 549.5161828495164,
200                 "mean_dur": 12.779446112779452,
201                 "max_dur": 47.66433099766434,
202                 "count_ratio": 1.075,
203                 "matched": 30,
204                 "local_only": 9,
205                 "gemini_only": 5,
206                 "splits": 1,
207                 "merges": 2,
208                 "mean_iou": 0.4837616679798798,
209                 "gem_rallies_in_merges": 4
210             }
211         },
212         "status": "ok"
213     },
214     {
215         "name": "mahadevpura-1",
216         "vid": "38bef75d78cb",
217         "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\local-noai\\output\\wasb\\mahadevpura-1__38bef75d78cb_wasb.csv",
218         "n_gem": 9,
219         "play_gem": 51.2,
220         "mean_gem": 5.688888888888889,
221         "variants": {
222             "baseline": {
223                 "n": 1,
224                 "play": 174.2742742742743,
225                 "mean_dur": 174.2742742742743,
226                 "max_dur": 174.2742742742743,

```

```

227         "count_ratio": 0.1111111111111111,
228         "matched": 0,
229         "local_only": 0,
230         "gemini_only": 0,
231         "splits": 0,
232         "merges": 1,
233         "mean_iou": 0.0,
234         "gem_rallies_in_merges": 9
235     },
236     "w1": {
237         "n": 2,
238         "play": 173.79045712379047,
239         "mean_dur": 86.89522856189524,
240         "max_dur": 101.88521855188523,
241         "count_ratio": 0.2222222222222222,
242         "matched": 0,
243         "local_only": 0,
244         "gemini_only": 0,
245         "splits": 0,
246         "merges": 2,
247         "mean_iou": 0.0,
248         "gem_rallies_in_merges": 9
249     },
250     "w1+w2": {
251         "n": 2,
252         "play": 173.79045712379047,
253         "mean_dur": 86.89522856189524,
254         "max_dur": 101.88521855188523,
255         "count_ratio": 0.2222222222222222,
256         "matched": 0,
257         "local_only": 0,
258         "gemini_only": 0,
259         "splits": 0,
260         "merges": 2,
261         "mean_iou": 0.0,
262         "gem_rallies_in_merges": 9
263     }
264 },
265 "status": "ok"
266 },
267 {
268     "name": "mahadevpura-2",
269     "vid": "038e0e0915a2",
270     "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\local-noai\\output\\wasb\\mahadevpura-2__038e0e0915a2_wasb.csv",
271     "n_gem": 39,
272     "play_gem": 266.0,
273     "mean_gem": 6.82051282051282,
274     "variants": {
275         "baseline": {
276             "n": 5,
277             "play": 605.9059059059059,
278             "mean_dur": 121.18118118118119,
279             "max_dur": 297.03036369703034,
280             "count_ratio": 0.1282051282051282,
281             "matched": 2,
282             "local_only": 0,
283             "gemini_only": 0,
284             "splits": 0,
285             "merges": 3,
286             "mean_iou": 0.2945165777074605,
287             "gem_rallies_in_merges": 37
288         },
289         "w1": {
290             "n": 40,

```

```

291         "play": 551.0844177510842,
292         "mean_dur": 13.777110443777104,
293         "max_dur": 43.3266599933267,
294         "count_ratio": 1.0256410256410255,
295         "matched": 12,
296         "local_only": 4,
297         "gemini_only": 0,
298         "splits": 4,
299         "merges": 5,
300         "mean_iou": 0.4204774017180335,
301         "gem_rallies_in_merges": 11
302     },
303     "w1+w2": {
304         "n": 40,
305         "play": 551.0844177510842,
306         "mean_dur": 13.777110443777104,
307         "max_dur": 43.3266599933267,
308         "count_ratio": 1.0256410256410255,
309         "matched": 12,
310         "local_only": 4,
311         "gemini_only": 0,
312         "splits": 4,
313         "merges": 5,
314         "mean_iou": 0.4204774017180335,
315         "gem_rallies_in_merges": 11
316     }
317 },
318 "status": "ok"
319 }
320 ]

```

14. assistant (2026-06-16T15:14:56.077Z)

Now I have the actual data. Let me carefully verify the claim against both the code and the data. Let me re-read the cited code lines precisely (the claim cites lines 305, 285, 292, 358-364).

The code I already read confirms:

- Line 296: ``prev = None`` on invisible frame (claim said line 285 ? slightly off, but the mechanism is correct; the actual reset is at line 296)
- Line 303: ``if velocity > velocity_thresh`` (claim said line 292 ? the velocity comparison is at line 303)
- Line 316: ``"end": group[-1][0] / fps + chunk_start`` (claim said line 305 ? the end computation is at line 316)
- Lines 358-364: the split-at-largest-gap logic, with line 364 keeping a gap-less group intact

Let me check the `velocity_thresh` default and the exact data points the claim cites.

15. user (2026-06-16T15:14:57.729Z)

```

261:         velocity_thresh: float = 0.02,
262:         merge_gap: float = 2.0,
263:         min_window_duration: float = 2.0,
265:         max_window_duration: float = 0.0,
266:         min_split_gap: float = 0.5,
272:         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
273:         ``merge_gap`` seconds of each other are grouped into one window; short
277:         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
282:         that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing fix
from
303:         if velocity > velocity_thresh:
310:         max_gap_frames = int(merge_gap * fps)
334:         if max_window_duration > 0:
336:         groups, int(max_window_duration * fps), int(min_split_gap * fps))

```

```
339:         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
```

16. assistant (2026-06-16T15:15:07.317Z)

Code mechanism is confirmed. Now let me verify the claim's core argument against the data, paying attention to whether the numbers actually support it.

Let me lay out what the claim asserts and check each piece:

****Claim's mechanism (code):**** Confirmed by the code:

- Active frame = visible AND velocity > 0.02 (line 303) ? "shuttle moving", not "rally"
- `prev=None` on invisible frame breaks trajectory (line 296)
- Window end = last in-flight frame `group[-1][0]/fps` (line 316)
- W1 cap splits ONLY at the largest internal inactivity gap ? min_split_gap, and leaves a gap-less group intact (lines 358-367, esp. 364)

****Claim's two cited data examples:****

1. mahadevpura-2 W1: mean 13.8s, count_ratio 1.03, "near-perfect 40 vs 39"
2. mahadevpura-1 W1: mean 86.9s, n=2 vs Gemini 9, mIoU 0.0

Let me verify these against the JSON:

- ****mahadevpura-2 W1****: `mean\_dur` = 13.777 (~13.8s ?), `count\_ratio` = 1.0256 (~1.03 ?), n=40 vs n_gem=39 ?. The "near-perfect 40 vs 39" refers to COUNT only. Note: matched=12, mean_iou=0.42, merges=5, gem_rallies_in_merges=11. So count is near-perfect but actual matching is weak. The claim says "near-perfect 40 vs 39" ? strictly about count, which is accurate.
 - ****mahadevpura-1 W1****: `mean\_dur` = 86.895 (~86.9s ?), n=2 ?, n_gem=9 ?, mean_iou=0.0 ?.
- Confirmed exactly.

Now let me critically test the ****causal claim****: that shuttle invisibility/slowdown between rallies is "the ONLY thing that ends or splits a window."

17. user (2026-06-16T15:15:08.321Z)

```
240         with open(csv_win_path, newline="") as f:
241             reader = csv.DictReader(f)
242             for row in reader:
243                 try:
244                     points.append(TrajectoryPoint(
245                         frame=int(row["Frame"]),
246                         visible=int(row["Visibility"]) == 1,
247                         x=float(row["X"]),
248                         y=float(row["Y"]),
249                     ))
250                 except (KeyError, ValueError):
251                     continue
252             points.sort(key=lambda p: p.frame)
253             return points
254
255     @staticmethod
256     def trajectory_to_action_windows(
257         points: List[TrajectoryPoint],
258         fps: float,
259         frame_width: float,
260         chunk_start: float = 0.0,
261         velocity_thresh: float = 0.02,
262         merge_gap: float = 2.0,
263         min_window_duration: float = 2.0,
264         chunk_index: Optional[int] = None,
265         max_window_duration: float = 0.0,
266         min_split_gap: float = 0.5,
267     ) -> List[Dict[str, Any]]:
268         """Convert a shuttle trajectory into candidate rally windows.
269
```

18. assistant (2026-06-16T15:15:24.289Z)

Now I have the full picture of the windowing logic. Let me reason carefully about the causal claim, testing it adversarially.

****Verifying the causal mechanism "invisibility/slowdown is the ONLY thing that ends or splits a window".****

Looking at the code precisely:

- A window (group) is BROKEN/ENDED when the gap between consecutive **active** frames exceeds ``max_gap_frames = merge_gap*fps`` (lines 326-332). An active frame requires visible AND `velocity>thresh`. So a window ends when there's a stretch `? 2.0s` with no in-flight shuttle `? i.e., the shuttle is either invisible OR moving slowly (velocity ? 0.02)`. This is exactly "shuttle goes invisible/slow." `?`
- W1 split: only at the largest internal inactivity gap `? min_split_gap`; a gap-less group stays intact (line 364). `?`

So the code mechanism strongly supports: window boundaries are created exclusively by gaps in shuttle in-flight activity (invisibility or sub-threshold velocity). There is no "rally"-aware signal. This is accurate.

****Now testing the cross-clip differential claim with the data `? the crux to refute:**`**

The claim says mahadevpura-1 stays "one blob" because it has "continuous low-level shuttle visibility/motion" with "no gap to cut on." Let me test this against the data:

- mahadevpura-1 W1: `n=2`, mean 86.9s, max 101.9s. So W1 DID split the baseline (`n=1`) into `n=2`. That means there WAS at least one inactivity gap (the cap split it once). But it remained 2 huge blobs (86.9s mean) `? far above the 12s cap ? because the remaining sub-groups had no internal gap ? min_split_gap (0.5s). This is consistent with the claim: the clip lacks the frequent dead-frame gaps needed to cut to rally scale.`

Let me sanity-check: baseline `n=1` (one 174s blob), W1 `n=2`. The split logic recursively splits at largest gap `? 0.5s`. It produced only 2 pieces from a 174s blob, both still `~87-102s`, meaning after the first cut, neither half had any internal gap `? 0.5s` long enough... actually it would keep splitting as long as gaps `? 0.5s` exist and the piece `> cap`. The fact it stopped at 2 means each resulting half had NO internal active-frame gap `? 0.5s`. That confirms "continuous low-level activity, no gap to cut on." `?`

Let me verify the contrast quantitatively across all clips to see if the "differs by clip" framing holds and isn't overstated.

19. user (2026-06-16T15:15:28.369Z)

clip	n_gem	W1_n	W1_mean	W1_max	cap=12?	cratio	mIoU
GX010128	39	78	7.8	16.6	True	2.00	0.53
GX010129	27	50	9.3	27.8	True	1.85	0.48
GX020125	11	18	14.1	40.1	True	1.64	0.43
mahadevpura-0	40	62	10.4	47.7	True	1.55	0.49
mahadevpura-1	9	2	86.9	101.9	True	0.22	0.00
mahadevpura-2	39	40	13.8	43.3	True	1.03	0.42

20. assistant (2026-06-16T15:15:41.774Z)

This is a critical observation for adversarial testing. ****Every single clip `? including the "well-behaved" ones ? has W1_max exceeding the 12s cap.**`** GX010128's max is 16.6s, GX010129's is 27.8s, mahadevpura-0's is 47.7s. So the cap fails to cut SOME windows to `?12s` in EVERY clip, not just mahadevpura-1. This confirms the gap-dependence mechanism is universal.

But let me test whether the claim is **overstated**. The claim frames it as a clean dichotomy: clips with "frequent" dead frames get good cut points (citing mahadevpura-2 W1 mean 13.8s, near-perfect count) vs. mahadevpura-1's "continuous activity, one blob." Let me scrutinize the mahadevpura-2 example the claim uses as its "good" exemplar.

mahadevpura-2 W1: mean 13.8s, count_ratio 1.03 (40 vs 39) `? but mean_iou is only **0.42**,`

matched=12 (of 39!), merges=5, gem_rallies_in_merges=11. So the count being "near-perfect" is somewhat coincidental ? only 12 of 39 Gemini rallies actually matched, and its mean (13.8s) is itself above the 12s cap. So mahadevpura-2 is NOT actually a clean example of the cap producing rally-scale windows; its mean exceeds the cap too. But the claim only cites it for "mean 13.8s, count_ratio 1.03, near-perfect 40 vs 39" ? all three of which are factually accurate in the data.

Let me check the claim's wording carefully: it says mahadevpura-2 "gets natural cut points... near-perfect 40 vs 39." The 40-vs-39 is COUNT. It does NOT claim high IoU. So the claim is using count-similarity as evidence of better cut behavior ? which is defensible but it's the weakest reading. The contrast in *mean window duration* (13.8s vs 86.9s) is the load-bearing comparison, and that's a 6.3x difference ? robustly supporting "behaves differently across clips."

Let me verify the GROUND-TRUTH and aggregate numbers cited aren't contradicted, and check the one possible overstatement: "the ONLY thing that ends or splits a window."

21. user (2026-06-16T15:15:45.947Z)

Gemini total: 165 (brief says 165)

baseline total: 48 (brief says 48)

W1 total: 250 (brief says 250)

W1+W2 total: 222 (brief says 222)

Per-video W1 mean_dur (brief: 7.8/9.3/14.1/10.4/86.9/13.8):

GX010128	7.8
GX010129	9.3
GX020125	14.1
mahadevpura-0	10.4
mahadevpura-1	86.9
mahadevpura-2	13.8

22. assistant (2026-06-16T15:16:03.126Z)

Every cited figure matches the data exactly. Now let me do the final adversarial test on the one phrase that could be overstated: *****that invisibility/slowdown is the ONLY thing that ends or splits a window.*****

Reading the code carefully, there are actually a few OTHER things that affect window boundaries:

1. ****Chunk boundaries**** (`chunk\_start`, `chunk\_index`) ? windows are computed per-chunk, so a chunk boundary could artificially end a window. But within `trajectory\_to\_action\_windows`, the only boundary-creating logic is the active-frame gap. Chunking is external.
2. ****min_window_duration**** filter (line 339) ? drops short windows, doesn't end/split them.
3. ****W2 net-crossings gate**** ? drops whole windows, doesn't end/split.
4. The W1 cap split: cuts at the largest gap. But note ? the cap split STILL requires a gap ? min_split_gap (0.5s) to exist. If a group exceeds the cap but has NO such gap, it stays intact (line 364). So even the cap is gated on the existence of an inactivity gap.

So within the windowing function itself, what ENDS a window (the merge_gap break) and what SPLITS a window (the cap, gated on a real gap) are BOTH driven exclusively by inactivity gaps (invisible OR sub-threshold-velocity frames). The claim's "ONLY thing" is accurate *within the windowing logic*. The mechanism does not consult any rally/score/serve signal. This is correct.

One last refutation attempt: is "slowdown" actually a window-ender? An active frame needs velocity > 0.02. If the shuttle is visible but moving slowly (e.g., at the top of a clear, or during a slow exchange), those frames are NOT active ? they contribute to a gap. So yes, "slow" frames break windows just like invisible frames. The claim's "invisible/slow" framing is precisely correct per line 303 + the gap logic.

I've now verified:

- ****Code mechanism****: fully confirmed (line numbers are slightly off ? the claim cites 285/292/305 but the actual lines are 296/303/316, and 358-364 is correct ? but every described behavior is exactly right).
- ****Every cited data point****: exact match (mahadevpura-2 13.8s/1.03/40-vs-39; mahadevpura-1 86.9s/n=2/9/mIoU 0.0; all per-video means; all aggregates).
- ****The causal claim****: the windowing has no rally-aware signal; boundaries arise only from

shuttle in-flight gaps; clips genuinely differ in mean window (6.3x between the two cited clips), and mahadevpura-1 stays a 2-blob because its sub-groups lack a 20.5s gap.

The one mild weakness: the "near-perfect 40 vs 39" exemplar is count-only ? mahadevpura-2's mean (13.8s) still exceeds the cap and its mIoU is only 0.42, so it's not a clean "rally-scale windows" success. But the claim only invokes it for count/mean, both accurate, and as a *relative* contrast to mahadevpura-1. The claim holds.

23. user (2026-06-16T15:16:15.748Z)

Structured output provided successfully

24. assistant (2026-06-16T15:16:26.186Z)

I've completed the adversarial verification. The claim holds.

****Verdict: HOLDS (true)****

I verified both the code mechanism and every cited data point, and tried hard to refute the claim. Findings:

****Code (`backend/pipeline/detectors/tracknet\_runner.py`):**** Every described behavior is exactly right:

- Active frame = visible AND velocity > `velocity\_thresh=0.02` (line 303 / default line 261) ? keys on "shuttle in flight," never "rally."
- `prev=None` on any invisible frame breaks the trajectory (line 296).
- Window end = `group[-1][0]/fps`, the last in-flight frame (line 316).
- Windows end only on an active-frame gap > `merge\_gap\*fps` (a 2s no-in-flight stretch); the W1 cap splits only at the largest internal inactivity gap and explicitly keeps a gap-less over-long group intact (line 364). So inactivity (invisibility OR sub-threshold "slow" velocity) is genuinely the only thing that ends or splits a window ? the "ONLY" qualifier survives scrutiny.

****Data (`output/local\_vs\_gemini\_6.json`):**** Every cited figure matches exactly ? mahadevpura-2 W1 (13.8s, count_ratio 1.03, 40 vs 39), mahadevpura-1 W1 (86.9s, n=2 vs 9, mIoU 0.0), all per-video means, and all aggregates (165/48/250/222).

****Refutation attempts that failed to break it:****

- The cited line numbers are slightly off (285/292/305 vs actual 296/303/316), but the mechanisms are correctly described.
- mahadevpura-2's "near-perfect 40 vs 39" is count-only (its mean 13.8s still exceeds the 12s cap, mIoU only 0.42, 12/39 matched) ? but the claim only invokes it for count/mean as a relative contrast, both accurate.
- W1_max exceeds the 12s cap in all six clips (16.6/27.8/40.1/47.7/101.9/43.3s), which reinforces the gap-dependence mechanism rather than refuting it.

One note for the record: the data file is at `C:\Users\avidu\Projects\badminton-highlight-indexer\output\local\_vs\_gemini\_6.json` (the main repo), not under the `w2-safeguard` worktree path.